

ME 371 - Data Driven Problem Solving

The Cooper Union for the Advancement of Science and Art

Fall 2025

Masoud Masoumi

Agenda

- Shallow Neural Networks
 - Intuition with example
 - Universal approximation theorem
 - Multivariate inputs and outputs
 - General case for shallow neural networks
 - Terminology
 - Optimization & Loss Functions
- Deep Neural Networks
- Size and Efficiency

Shallow Neural Networks

Shallow Neural Networks

- **Shallow neural networks** are functions map multivariate inputs $\mathbf{X}$ to multivariate outputs $\mathbf{y}$.
- Let's first focus on a simple example of mapping a scalar input x to scalar output y with 10 parameters

$W = [w_0, w_1, w_2, w_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}]$,
resulting in:

$$y = f[x, W] = w_0 + w_1 a[\theta_{10} + \theta_{11}x] + w_2 a[\theta_{20} + \theta_{21}x] + w_3 a[\theta_{30} + \theta_{31}x]$$

- We can break down this equation into **three** parts:
 - i. Computing three linear functions
 - ii. Passing the results through an *activation function*, $a[\cdot]$
 - iii. Weighting the three resulting activations with w_1 , w_2 , and w_3 .
- An example of an activation function is *rectified linear unit* or ReLU:

$$a[z] = \text{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$

Shallow Neural Networks (Con't)

- We can break down the three linear equations by introducing intermediate quantities

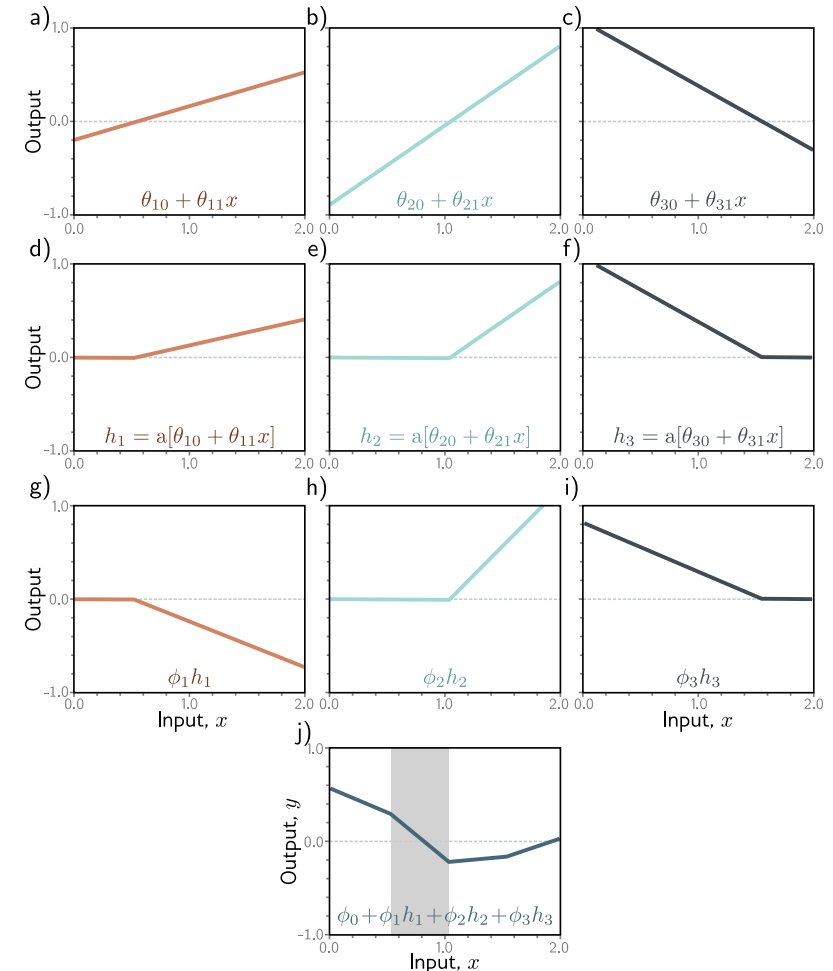
$$\begin{cases} h_1 = a[\theta_{10} + \theta_{11}x] \\ h_2 = a[\theta_{20} + \theta_{21}x] \\ h_3 = a[\theta_{30} + \theta_{31}x] \end{cases}$$

We refer to these quantities as *hidden units*.

- We then compute the output by combining these hidden units with a linear function

$$y = w_0 + w_1h_1 + w_2h_2 + w_3h_3$$

- See the interactive plot in [this link](#) (1D shallow network - ϕ 's are our w 's)



In the shaded region, h_2 is inactive (clipped), but h_1 and h_3 are both active.

TensorFlow Implementation

TensorFlow provides activations as functions and layers

```
import tensorflow as tf

# creating an input
x = tf.constant([-2.0, -1.0, 0.0, 1.0, 2.0])

# As functions
relu_output = tf.nn.relu(x)           # [0, 0, 0, 1, 2]
sigmoid_output = tf.nn.sigmoid(x)     # [0.12, 0.27, 0.5, 0.73, 0.88]
```

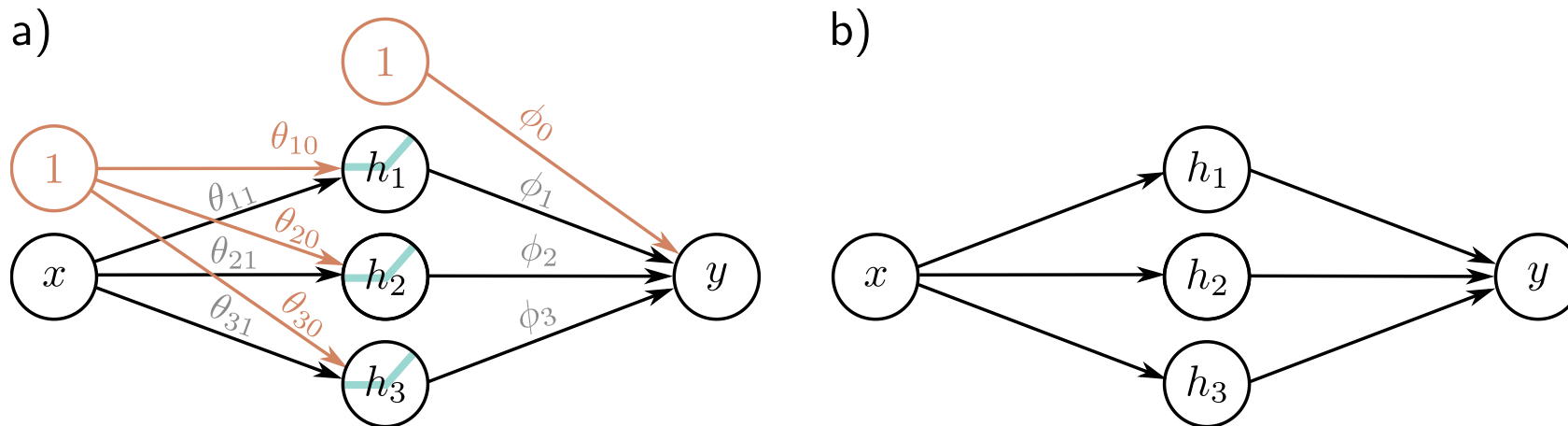
NOTE: Review Part 1 to implement and compare various activation functions, such as "relu", "Sigmoid", etc.

Shallow Neural Networks (Con't)

In the left figure, which is a visualization of our exampl network, the input is on the left, the hidden units are in the middle, and the output is on the right.

Each connection represents one of the ten parameters. Here, each of the **ten** arrows represents a parameter - intercepts in orange and slopes in black.

To simplify this representation, we do not typically draw the intercept parameters, so this network is usually depicted as shown in figure right.



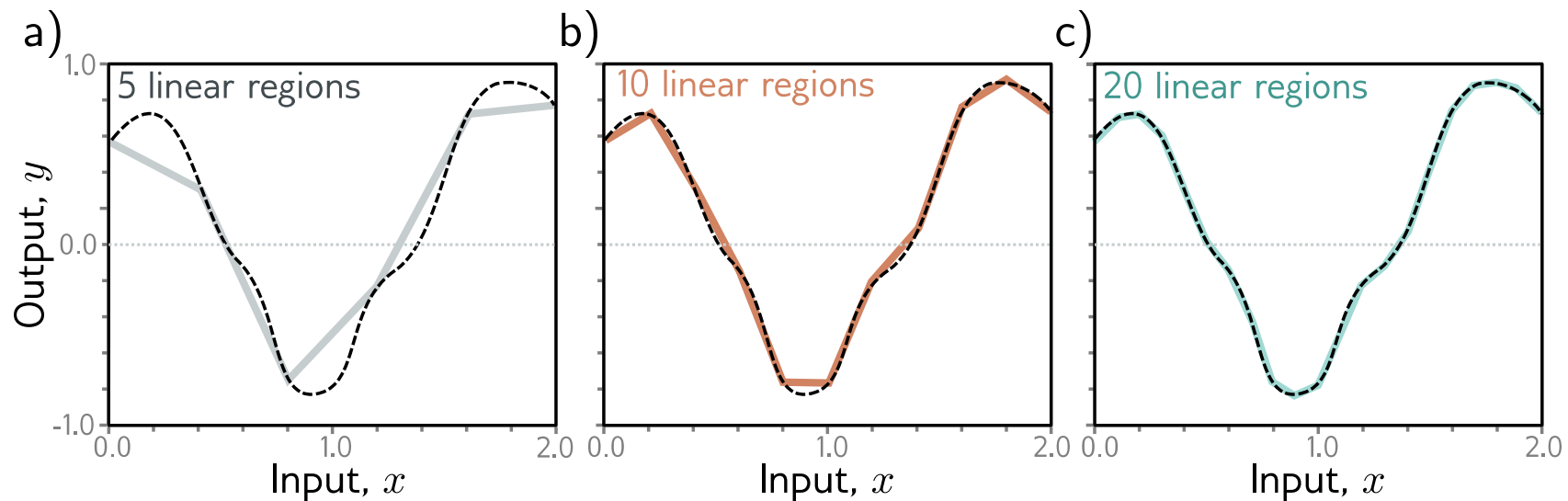
Universal Approximation Theorem

Let's now generalize single-input single-output slightly and consider the case with D hidden units where the d^{th} hidden unit is $h_d = a[\theta_{d0} + \theta_{d1}x]$, which results in a linearly combined output as

$$y = w_0 + \sum_{d=1}^D w_d h_d$$

The number of hidden units in a shallow network is a measure of the *network capacity* (remember *model capacity* from before?).

The **universal approximation theorem** proves that for any continuous function, there exists a shallow network that can approximate this function to any specified precision.



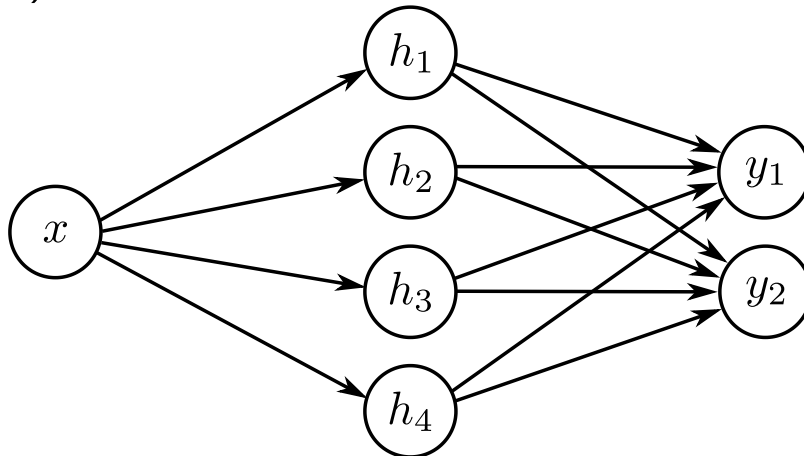
Multivariate Inputs and Outputs (1 input - 2 outputs)

To extend the network to multivariate outputs y , we simply use a different linear function of the hidden units for each output.

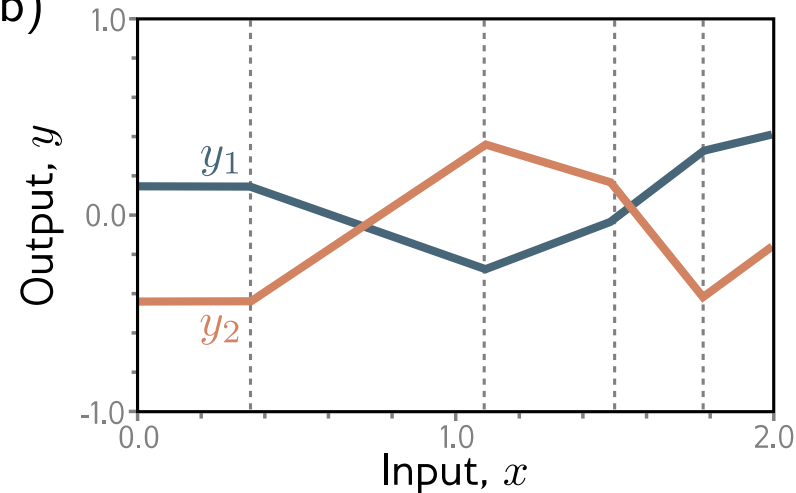
A network with a scalar input x , four hidden units h_1 , h_2 , h_3 , and h_4 , and a 2D multivariate output $y = [y_1, y_2]^T$ would be defined as:

$$\begin{cases} h_1 = a[\theta_{10} + \theta_{11}x] \\ h_2 = a[\theta_{20} + \theta_{21}x] \\ h_3 = a[\theta_{30} + \theta_{31}x] \\ h_4 = a[\theta_{40} + \theta_{41}x] \end{cases} \rightarrow \begin{cases} y_1 = w_{10} + w_{11}h_1 + w_{12}h_2 + w_{13}h_3 + w_{14}h_4 \\ y_2 = w_{20} + w_{21}h_1 + w_{22}h_2 + w_{23}h_3 + w_{24}h_4 \end{cases}$$

a)



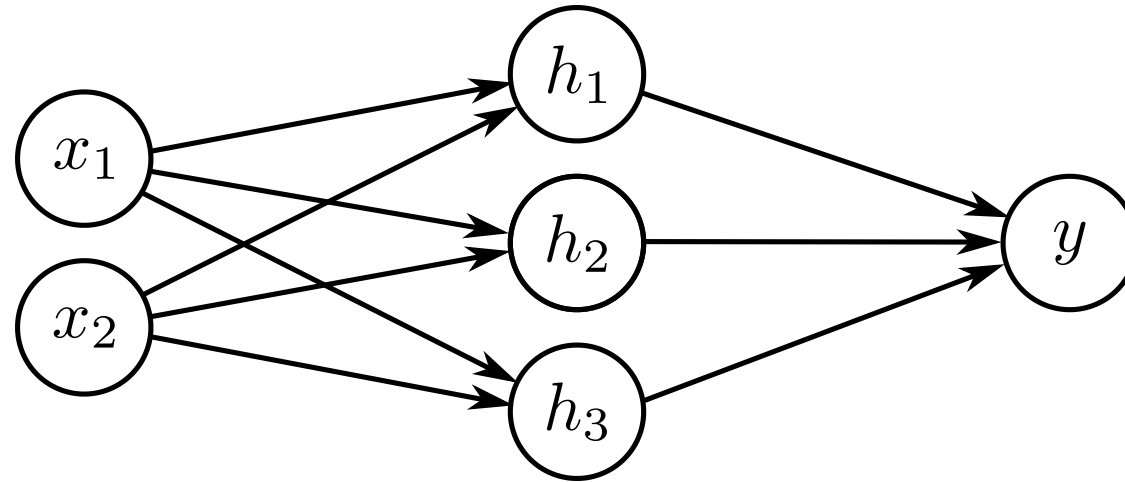
b)



Multivariate Inputs and Outputs (2 inputs - 1 output)

A network with two inputs $x = [x_1, x_2]^T$ and a scalar output y with three hidden units can be defined by:

$$\begin{cases} h_1 = a[\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2] \\ h_2 = a[\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2] \\ h_3 = a[\theta_{30} + \theta_{31}x_1 + \theta_{32}x_2] \end{cases} \rightarrow y = w_0 + w_1h_1 + w_2h_2 + w_3h_3$$



See an interactive visualization in [this link](#) (2D shallow network - ϕ 's are our w 's).

NOTE: Brightness indicates function output. For example, in top left panel, the brightness represents $\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2$. Thin lines are contours. Each plane is clipped by the ReLU activation function (Cyan lines).

Output Layer

- The output layer determines how your network makes predictions
- For binary classification:
 - Single output neuron
 - Sigmoid activation: squashes output to [0,1]
 - Output > 0.5 → Class 1
 - Output ≤ 0.5 → Class 0
- For multi-class classification:
 - One neuron per class (Highest probability wins)
 - Softmax activation: converts outputs to probabilities
 - $P(y=y_1 | x) = \frac{e^{z_1}}{\sum_{c=1}^c e^{z_c}}$
 - $P(y=y_2 | x) = \frac{e^{z_2}}{\sum_{c=1}^c e^{z_c}}$
 - ...
 - $P(y=y_c | x) = \frac{e^{z_c}}{\sum_{c=1}^c e^{z_c}}$

Building Networks with Keras Sequential API

Network has 13 trainable parameters:

- $W1: (2 \times 3) = 6$ weights, $b1: 3$ biases
- $W2: (3 \times 1) = 3$ weights, $b2: 1$ bias

```
from tensorflow.keras import models, layers

# Method 1: Layer by layer
model = models.Sequential()
model.add(layers.Input(shape=(2,)))           # Input: 2 features
model.add(layers.Dense(3, activation='relu')) # Hidden: 3 units
model.add(layers.Dense(1, activation='sigmoid')) # Output: 1 unit
# Method 2: All at once (preferred)
model = models.Sequential([
    layers.Input(shape=(2,)),
    layers.Dense(3, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

Building Networks with Keras Sequential API

```
# Keras can generate architecture diagrams
tf.keras.utils.plot_model(
    model,
    to_file='model.png',
    show_shapes=True,
    show_layer_names=True
)

# Text summary
model.summary()

# Inspect specific layer
layer = model.get_layer('dense')
weights, biases = layer.get_weights()
print(f"Weights shape: {weights.shape}")    # (2, 3)
print(f"Biases shape: {biases.shape}")      # (3,)
```

NOTE: Complete Part 2 of IN-CLASS ASSIGNMENT.

Shallow Networks: General Case

For a shallow neural network $y=f[x,W]$ that maps a multi-dimensional input $\mathbf{X}$ to a multi-dimensional output $\mathbf{y}$ using $\mathbf{h}$ hidden units, where each hidden unit is computed as

$$h_d = a[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i]$$

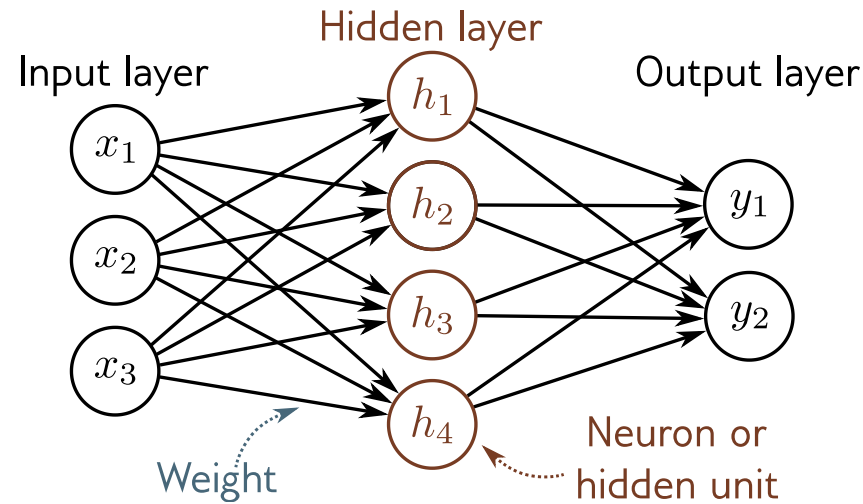
which will be combined linearly to create the output

$$y_j = w_{j0} + \sum_{d=1}^D w_{jd} h_d$$

where $a[.]$ is a *non-linear* activation function. The model has parameters $W=[\theta_{..}, w_{..}]$

NOTE: The activation function permits the model to describe nonlinear relations between input and the output, and as such, it must be nonlinear itself; with no activation function, or a linear activation function, the overall mapping from input to output would be restricted to be linear.

Terminology



- Any neural network with at least one hidden layer is also called a **multi-layer perceptron**, or **MLP** for short.
- Networks with one hidden layer are sometimes referred to as **shallow neural networks**.
- Networks with multiple hidden layers are referred to as **deep neural networks**.
- Neural networks in which the connections form an acyclic graph (i.e., a graph with no loops) are referred to as **feed-forward networks**.
- If every element in one layer connects to every element in the next, the network is **fully connected**.
- The offset parameters (not shown in the figure) are called **biases**.

Loss Function

Loss Functions for Classification

- **Binary Cross-Entropy:**

- Same formula as Log Loss!
- $BCE = - \sum (y_i \log(p_i) + (1 - y_i) \log(1 - p_i))$
- Perfect for binary classification
- Why it works:
 - Heavily penalizes confident wrong predictions
 - Minimal penalty for correct predictions

- **Common Loss Functions in TensorFlow:**

- Binary classification: `binary_crossentropy`
- Multi-class: `sparse_categorical_crossentropy`
- Regression: `mean_squared_error`

Binary Cross-Entropy in TensorFlow

```
# Method 1: String (during model compilation)
model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# Method 2: Function object (more control)
loss_fn = tf.keras.losses.BinaryCrossentropy()
model.compile(loss=loss_fn, metrics=['accuracy'])

# Computing loss manually
y_true = tf.constant([[1.0, 0.0, 1.0]])
y_pred = tf.constant([[0.9, 0.1, 0.8]])

loss = loss_fn(y_true, y_pred)
print(f"Loss: {loss.numpy():.4f}") # Low loss = good predictions
```

NOTE: Complete Part 3 of IN-CLASS ASSIGNMENT.

Optimization (Minimization)

Optimization

- To fit a model, we need a training set x_i, y_i of input/output pairs. We seek parameters W for the model $f[x_i, W]$ that map the inputs x_i to the outputs y_i as well as possible.
- To this end, we define a loss function $L[W]$ that returns a single number that quantifies the mismatch in this mapping. The goal of an *optimization algorithm* is to find parameters $\hat{W}$ that minimize the loss:

$$\hat{W} = \operatorname{argmin}[L[W]]$$

- There are many families of optimization algorithms, but the standard methods for training neural networks are iterative. These algorithms initialize the parameters heuristically and then adjust them repeatedly in such a way that the loss decreases.

Gradient Descent

- The simplest method in this class is *gradient descent*. This starts with initial parameters $W = [w_0, w_1, \dots, w_N]^T$ and iterates two steps:

- Step 1: Compute the derivatives of the loss with respect to the parameters:

$$\frac{\partial L}{\partial W} = \begin{bmatrix} \frac{\partial L}{\partial w_0} \\ \frac{\partial L}{\partial w_1} \\ \cdot \\ \cdot \\ \cdot \\ \frac{\partial L}{\partial w_N} \end{bmatrix}$$

- Step 2: Update the parameters according to the rule:

$$W \rightarrow W - \alpha \cdot \frac{\partial L}{\partial W}$$

where the positive scalar α determines the magnitude of the change. This is called *learning rate*.

Training Concepts

- **Epochs:** Complete pass through entire training dataset
 - More epochs → More learning opportunities
 - Too many → Overfitting
 - Too few → Underfitting
- **Batch Size:** Number of samples processed before model update
 - Larger batch → Stable but slower learning
 - Smaller batch → Faster but noisier learning
 - Common sizes: 32, 64, 128, 256

Types of Gradient Descent

- **Batch Gradient Descent:**

- Uses entire dataset for each update
- $w = w - \alpha \frac{\partial J}{\partial w}$ with $J = \frac{1}{m} \sum_{i=1}^m L(y_i, \hat{y}_i)$
- Stable but very slow for large datasets

- **Stochastic Gradient Descent (SGD):**

- Uses single sample for each update
- $w = w - \alpha \frac{\partial L_i}{\partial w}$
- Noisy but faster updates and it can escape local minima.

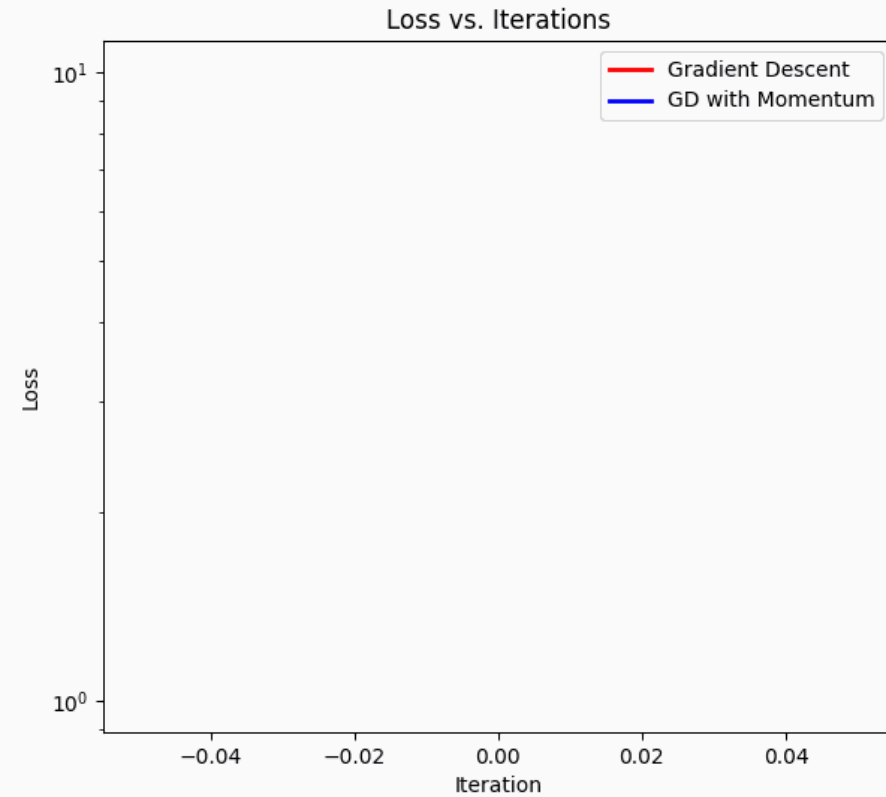
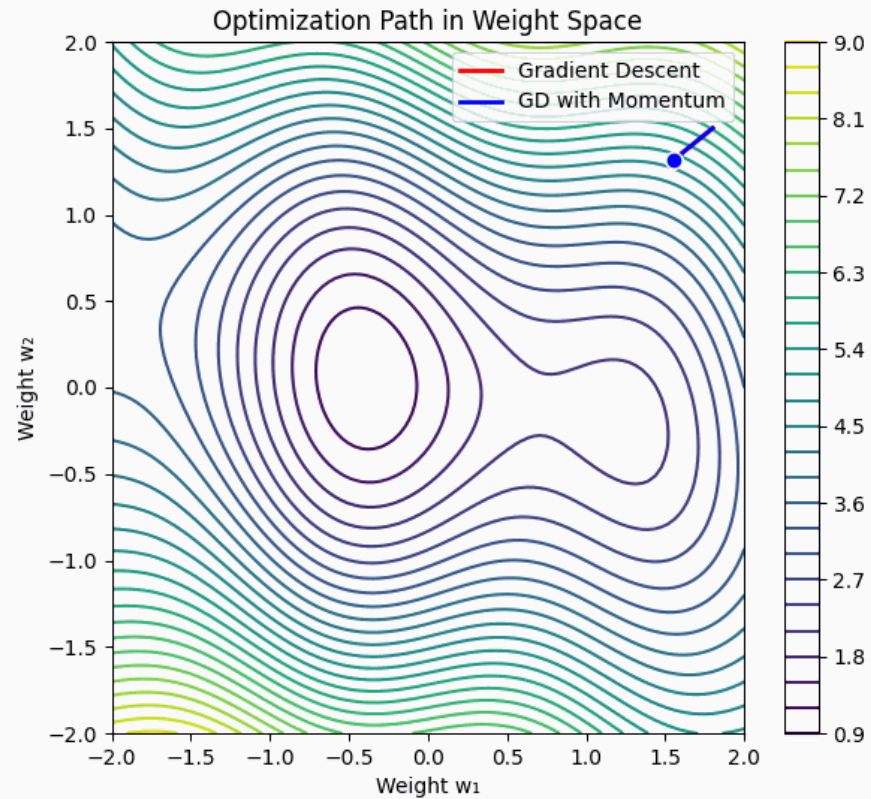
- **Mini-batch Gradient Descent:**

- Uses batch_size samples per update
- $w = w - \alpha \frac{1}{b} \sum_{i=1}^b \frac{\partial L_i}{\partial w}$
- b : batch size (e.g., 32, 64, 128)

- **Momentum:**

- Adds "velocity" to updates
- $\hat{v} = \beta v - \alpha \frac{\partial J}{\partial w}$
- $w = w + \hat{v}$
- β : momentum term (typically 0.9)
- Helps escape local minima
- Can be applied to any GD variant

Batch Gradient Descent with and without Momentum



Training in TensorFlow

```
# 1. Prepare data
X_train, y_train = ... # Your training data
X_val, y_val = ...     # Your validation data

# 2. Build model
model = models.Sequential([ layers.Input(shape=(2,)),
                             layers.Dense(3, activation='relu'),
                             layers.Dense(1, activation='sigmoid')])

# 3. Compile (define loss and optimizer)
model.compile(optimizer='adam',                    # Optimizer handles backprop!
              loss='binary_crossentropy',
              metrics=['accuracy'])

# 4. Train (one line!)
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val), epochs=100, batch_size=32
)
```

NOTE: Complete Part 4 and 5 of IN-CLASS ASSIGNMENT.

Advanced Optimizers

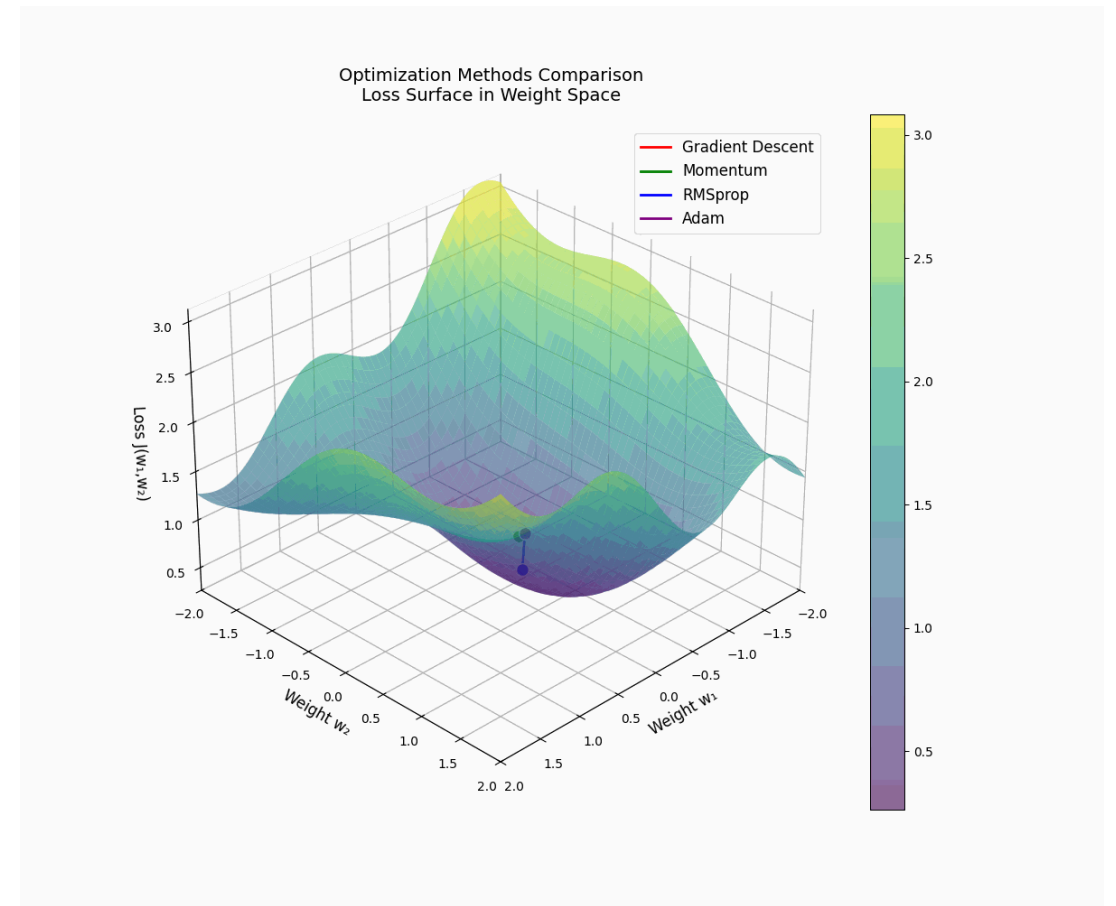
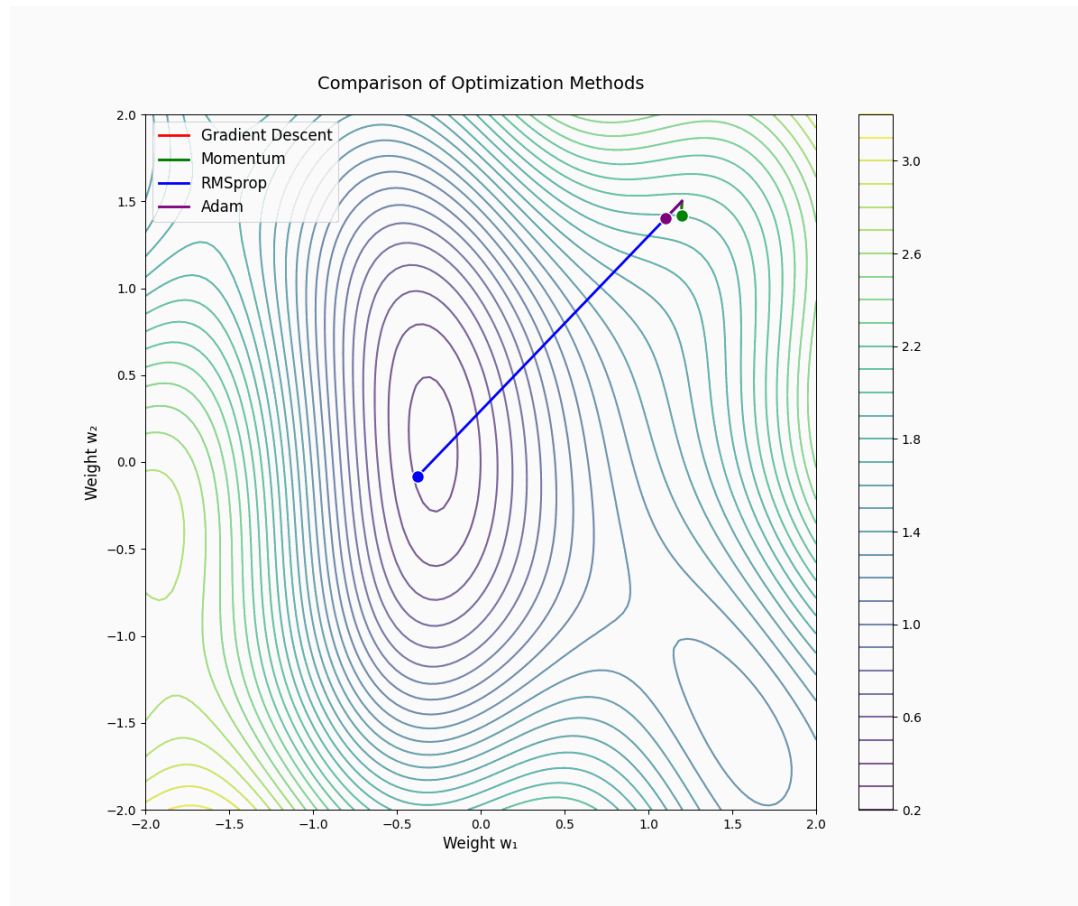
- **RMSprop:**

- Adapts learning rate
- $\hat{s} = \rho s + (1 - \rho) \left(\frac{\partial J}{\partial w} \right)^2$
- $w = w - \alpha \frac{\partial J}{\partial w} / \sqrt{\hat{s} + \epsilon}$
- ρ : decay rate (typically 0.9)

- **Adam** (Adaptive Moment Estimation):

- Combines Momentum and RMSprop
- $\hat{m} = \beta_1 m + (1 - \beta_1) \frac{\partial J}{\partial w}$ [momentum]
- $\hat{v} = \beta_2 v + (1 - \beta_2) \left(\frac{\partial J}{\partial w} \right)^2$ [RMSprop]
- $w = w - \alpha \hat{m} / \sqrt{\hat{v} + \epsilon}$
- β_1, β_2 : decay rates (0.9, 0.999)

Optimizations Methods - Comparisons



Optimizers in TensorFlow

```
# SGD
model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
              loss='binary_crossentropy')

# SGD with Momentum
model.compile(
    optimizer=tf.keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9),
    loss='binary_crossentropy')

# Adam (most popular - adaptive learning rate)
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy')

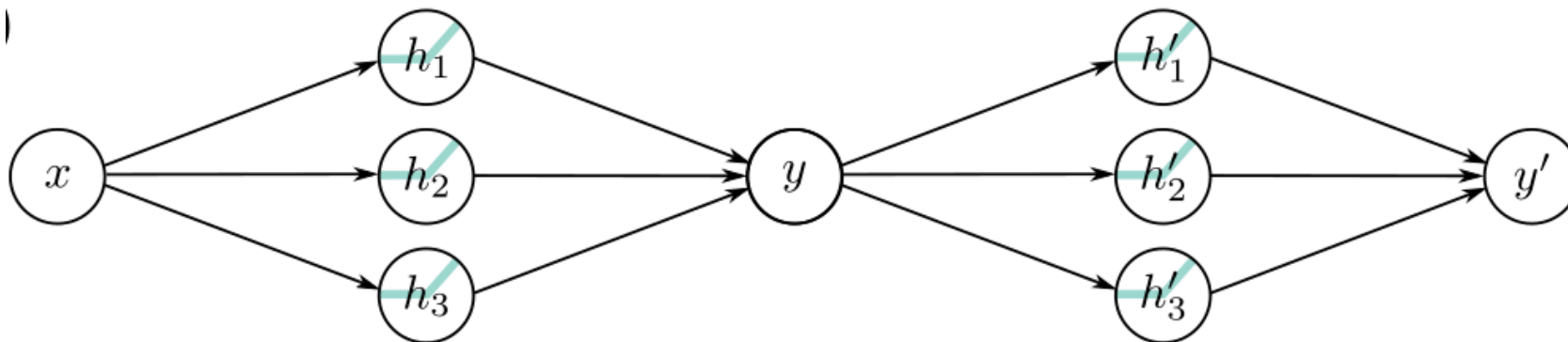
# RMSprop
model.compile(
    optimizer=tf.keras.optimizers.RMSprop(learning_rate=0.001),
    loss='binary_crossentropy')
```

NOTE: Complete Part 6 of IN-CLASS ASSIGNMENT.

Deep Neural Networks

Deep Neural Networks

- As the number of hidden units increases, shallow neural networks improve their descriptive power. They can describe arbitrarily complex functions in high dimensions.
- However, it turns out that for some functions, the required number of hidden units is impractically large.
- Hence, from a practical standpoint, Deep networks can be used to describe a broader family of functions.
- Let's consider composing **two shallow networks** so the output of the first becomes the input of the second.



Deep Neural Networks (Con't)

- First network:

$$\begin{cases} h_1 = a[\theta_{10} + \theta_{11}x] \\ h_2 = a[\theta_{20} + \theta_{21}x] \\ h_3 = a[\theta_{30} + \theta_{31}x] \end{cases} \rightarrow y = w_0 + w_1h_1 + w_2h_2 + w_3h_3$$

- Second network:

$$\begin{cases} h'_1 = a[\theta'_{10} + \theta'_{11}y] \\ h'_2 = a[\theta'_{20} + \theta'_{21}y] \\ h'_3 = a[\theta'_{30} + \theta'_{31}y] \end{cases} \rightarrow y' = w'_0 + w'_1h'_1 + w'_2h'_2 + w'_3h'_3$$

- The overall effect is that the function defined by the second network is duplicated three times to create nine linear regions.
- See the interactive plot [here](#) (under Concatenating networks).

Deep Neural Networks (Con't)

Now, let's show that this is a special case of a deep network with two hidden layers.

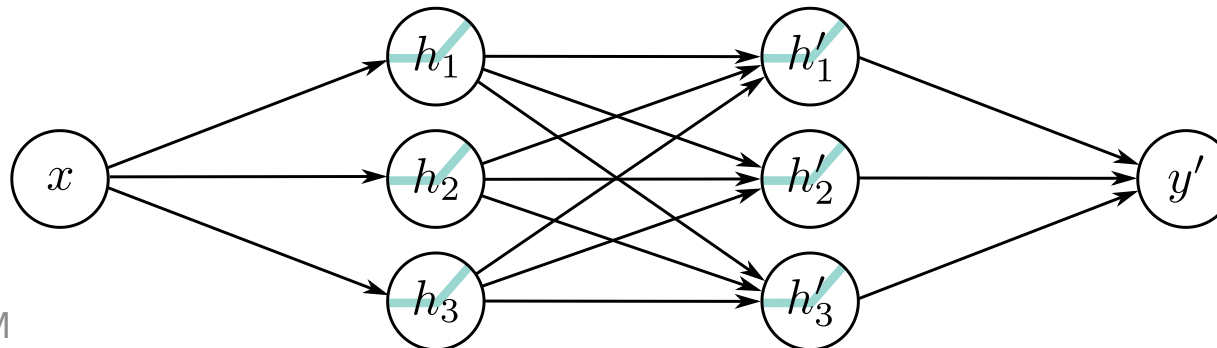
$$\begin{cases} h'_1 = a[\theta'_{10} + \theta'_{11}y] \\ h'_2 = a[\theta'_{20} + \theta'_{21}y] \\ h'_3 = a[\theta'_{30} + \theta'_{31}y] \end{cases} \rightarrow \begin{cases} h'_1 = a[\theta'_{10} + \theta'_{11}w_0 + \theta'_{11}w_1h_1 + \theta'_{11}w_2h_2 + \theta'_{11}w_3h_3] \\ h'_2 = a[\theta'_{20} + \theta'_{21}w_0 + \theta'_{21}w_1h_1 + \theta'_{21}w_2h_2 + \theta'_{21}w_3h_3] \\ h'_3 = a[\theta'_{30} + \theta'_{31}w_0 + \theta'_{31}w_1h_1 + \theta'_{31}w_2h_2 + \theta'_{31}w_3h_3] \end{cases}$$

which we can re-write as

$$\begin{cases} h'_1 = a[\psi_{10} + \psi_{11}h_1 + \psi_{12}h_2 + \psi_{13}h_3] \\ h'_2 = a[\psi_{20} + \psi_{21}h_1 + \psi_{22}h_2 + \psi_{23}h_3] \\ h'_3 = a[\psi_{30} + \psi_{31}h_1 + \psi_{32}h_2 + \psi_{33}h_3] \end{cases} \rightarrow y' = w'_0 + w'_1h'_1 + w'_2h'_2 + w'_3h'_3$$

with $\psi_{10} = \theta'_{10} + \theta'_{11}w_0$, $\psi_{11} = \theta'_{11}w_1$, $\psi_{12} = \theta'_{11}w_2$,

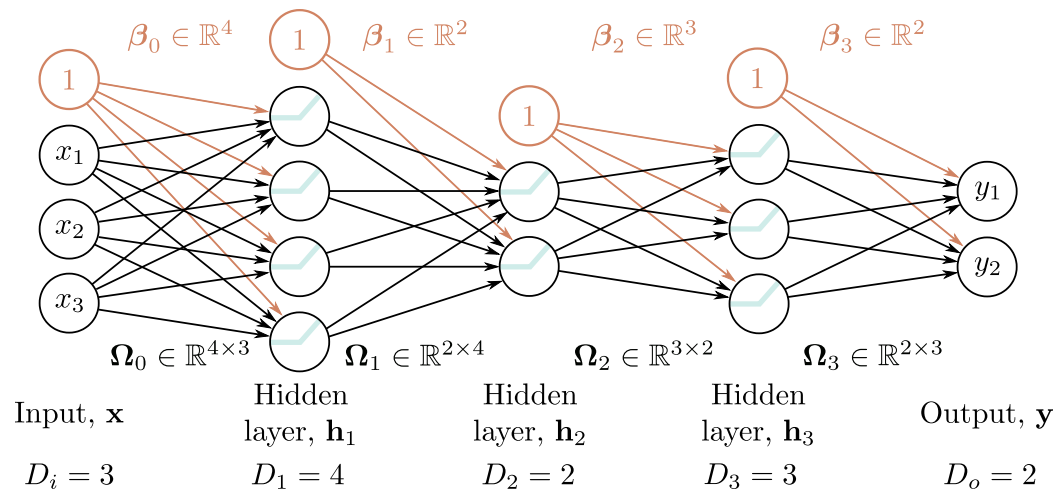
So this is just a network with one input, one output, and two hidden layers. See interactive visualization [here](#) (section Deep network computation.)



Deep Neural Networks (Con't)

This notation becomes cumbersome for networks with many layers. Therefore, a matrix notation approach is typically adopted, where

- $\mathbf{K}$ denotes the number of layers
- Number of hidden units in each layer denoted as $D_1, D_2, \dots, D_K$
- Vector of hidden units at layer k is noted as $\mathbf{h}_k$
- Vector of biases (intercepts) contributing to hidden layer $k + 1$ defined as β_k
- Weights (slopes) applied to the k^{th} layer contributing to the $(k + 1)^{th}$ layer as Ω_k .



$$\begin{cases} h_1 = a[\beta_0 + \Omega_0 x] \\ h_2 = a[\beta_1 + \Omega_1 h_1] \\ h_3 = a[\beta_2 + \Omega_2 h_2] \\ \vdots \\ h_k = a[\beta_{K-1} + \Omega_{K-1} h_{K-1}] \end{cases}, \quad y = \beta_K + \Omega_K h_K$$

Deep NN in TensorFlow

```
# Shallow: 2 → 10 → 1
shallow_model = models.Sequential([
    layers.Input(shape=(2,)), layers.Dense(10, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Deep: 2 → 10 → 10 → 1
deep_model = models.Sequential([ layers.Input(shape=(2,)),
    layers.Dense(10, activation='relu'),
    layers.Dense(10, activation='relu'), # Additional hidden layer
    layers.Dense(1, activation='sigmoid')
])

# Very Deep: 2 → 20 → 15 → 10 → 5 → 1
very_deep_model = models.Sequential([ layers.Input(shape=(2,)),
    layers.Dense(20, activation='relu'),
    layers.Dense(15, activation='relu'),
    layers.Dense(10, activation='relu'),
    layers.Dense(5, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

Deep NN in TensorFlow

Creating architectures programmatically

```
def build_deep_network(layer_dims):  
    """layer_dims: [input, hidden1, hidden2, ..., output]"""  
    model = models.Sequential()  
    model.add(layers.Input(shape=(layer_dims[0],)))  
  
    # Add hidden layers  
    for units in layer_dims[1:-1]:  
        model.add(layers.Dense(units, activation='relu'))  
  
    # Add output layer  
    model.add(layers.Dense(layer_dims[-1], activation='sigmoid'))  
  
    return model  
  
# Usage  
model = build_deep_network([2, 20, 15, 10, 1])
```

NOTE: Review Part 7 and 8 of IN-CLASS ASSIGNMENT.

Size and Efficiency

Size and Efficiency

- Depth Efficiency
 - Both deep and shallow networks can model arbitrary functions, but some functions can be approximated much more efficiently with deep networks.
 - A shallow network with exponentially more hidden units can achieve an equivalent approximation to that of a deep network. This phenomenon is referred to as the *depth efficiency* of neural networks.
- Large, structured inputs
 - Fully connected networks are not practical for large structured inputs like images, where the input might comprise $\sim 10^6$ pixels.
 - There is no point in independently learning to recognize the same object at every possible position in the image.
- Training and generalization
 - It is usually easier to train moderately deep networks than to train shallow ones.
 - Deep neural networks also seem to generalize to new data better than shallow ones. This phenomenaon is not well understood.